



HW #4 (Bonus): The Mysterious Ticking Noise

Faculty of Computer and Information Science, Ben-Gurion University of the Negev

Spring 2026

Submission in pairs (groups of two students)



Reference video: youtube.com/watch?v=Tx1XIm6q4r4

Overview

In this bonus assignment you will implement a Harry Potter-themed countdown simulation that demonstrates two classical object-oriented design patterns working in tandem: the **Observer pattern** and the **State pattern**.

A **PipeBomb** counts down from a configurable number of ticks to zero, notifying every registered observer on each tick. Each observer is a **state machine entity** that cycles through a fixed sequence of named states, printing a state's name the moment it enters that state. When the countdown expires the bomb prints **BOOM!** and the simulation ends.

Concepts practiced

- Observer pattern (subject/observer interface design)
- State pattern (cyclic state machines driven by a tick counter)
- Functional interfaces and lambdas
- Java records as immutable value objects
- Varargs, defensive copying, and constructor-time precomputation
- Fail-fast validation
- Composition over inheritance

Read the entire assignment before starting.

Component Specifications

The system consists of five components. All classes belong to the package `com.mysteryticking`.

TickListener

`TickListener` is the **observer interface**. It must be annotated `@FunctionalInterface` and declare exactly one abstract method.

API Specification

```
@FunctionalInterface
public interface TickListener {
    void onTick(int tick);
}
```

Because `TickListener` is a functional interface, observers may be supplied as lambdas without defining a concrete class.

Observable

`Observable` is the **subject interface**. It defines the two operations any observable subject must support.

API Specification

```
public interface Observable {
    void register(TickListener listener);
    void notifyListeners(int tick);
}
```

State

`State` is an immutable value object representing one step in a state machine's cycle. Implement it as a Java **record**.

API Specification

```
public record State(String name, int cost) { ... }
```

Components

- **name** — the string printed when this state is entered. Must not be `null`.
- **cost** — the number of ticks this state occupies within one cycle. Must be ≥ 1 .

Validation

The compact canonical constructor must throw `IllegalArgumentException` with a descriptive message if `name` is `null` or `cost < 1`.

PipeBomb

PipeBomb is the **concrete subject**. It implements `Observable` and drives the tick loop.

API Specification

```
public class PipeBomb implements Observable {
    public PipeBomb(int countdown)
    @Override public void register(TickListener listener)
    @Override public void notifyListeners(int tick)
    public void start()
}
```

Fields

- `private final int countdown`
- `private final List<TickListener> listeners` — declaration type must be `List` (not `ArrayList`); backed by `ArrayList`.

Behavior

- `PipeBomb(int countdown)` — throws `IllegalArgumentException` if `countdown < 1`.
- `register(TickListener)` — appends the listener; registration order determines notification order within each tick.
- `notifyListeners(int tick)` — calls `onTick(tick)` on every registered listener, in registration order.
- `start()` — runs a loop from tick 1 to `countdown` (inclusive), calling `notifyListeners` on each iteration. After the loop, prints `BOOM!` to standard output.

StateMachineEntity

StateMachineEntity is the **concrete observer**. It implements `TickListener` and models an entity that cycles through a fixed sequence of `States`, printing a state's name exactly once — the moment it enters that state.

API Specification

```
public class StateMachineEntity implements TickListener {
    public StateMachineEntity(String entityName,
                              int startTick,
                              State... states)
    @Override public void onTick(int tick)
    @Override public String toString()
}
```

Inputs (all private final)

- `String entityName` — name used in `toString()`.
- `int startTick` — the entity is silent for all ticks strictly before this value.



Principles of Object-Oriented Programming

- A sequence of **States**, stored as `List<State>`. The sequence must be copied defensively before being stored.

Any additional fields needed to implement the tick logic (e.g. a cached cycle length, cumulative costs, a running counter) are left to your discretion.

Constructor

- Accepts varargs (`State... states`).
- Throws `IllegalArgumentException` if no states are provided.
- Stores a defensive copy of the sequence.

Tick mechanics

Let *cycleLength* be the sum of all state costs in the sequence. On each call to `onTick(tick)`:

- If `tick < startTick`, the entity is silent.
- Otherwise, let $posInCycle = (tick - startTick) \bmod cycleLength$. The entity has just *entered* state *i* iff *posInCycle* equals the sum of the costs of states $0, 1, \dots, i - 1$. On entry, print `states.get(i).name()` if it is non-empty.
- If *posInCycle* does not coincide with the start of any state, the entity is mid-state: do nothing.

The algorithm and data structures used to detect state entry are your choice — only the observable behavior is specified.

`toString()` returns `entityName`.

Wiring the Simulation

The entry point creates one `PipeBomb` (default countdown **160**, overridable by an integer **command-line argument**) and registers six listeners in the order shown below.

Entity	startTick	State sequence (name, cost)	cycleLen
Snape	1	("Snape",2), ("Snape",2), ("Severus Snape",4)	8
Dumbledore	16	("Dumbledore",8)	8
Ron	33	("Ron",2), ("Ron",2), ("Ron Weasley",4)	8
Hermione	49	("Hermione",4), ("Hermione",4), ("Hermione",2), ("Hermione",2), ("Hermione",4)	16
Harry Potter	65	("Harry Potter",1), ("Harry Potter",1), ("OoOohhhh",2), ("Harry Potter",1), ("Harry Potter",1), ("YeeeEeaaH",2)	8
CycleSeparator	—	Lambda: print a blank line when <code>tick % 8 == 0</code>	—

The **CycleSeparator** must be registered **last** so it always fires after all entity output for a given tick. It must be implemented as a **lambda** — no separate class.

Expected Output

Mechanics summary

- A state's name is printed **once, when entered** — not on every tick it remains active.

- When multiple entities fire on the same tick they print in *registration order*.
- A blank line appears after every tick t such that $t \bmod 8 = 0$.
- After tick countdown, BOOM! is printed and the program exits.

Sample output (countdown = 24)

The output below is the exact result of running `java -jar hw4.jar 24`.

Expected Output (countdown = 24)

```
Snape
Snape
Severus Snape

Snape
Snape
Severus Snape
Dumbledore

Snape
Snape
Severus Snape
Dumbledore

BOOM!
```

Trace of selected ticks

- **t=1:** Snape's $posInCycle = (1 - 1) \bmod 8 = 0$, coincides with the start of state 0 $\rightarrow$ **Snape**
- **t=3:** $posInCycle = 2$, coincides with the start of state 1 $\rightarrow$ **Snape**
- **t=5:** $posInCycle = 4$, coincides with the start of state 2 $\rightarrow$ **Severus Snape**
- **t=8:** Snape's $posInCycle = 7$, mid-state — silent. CycleSeparator fires: blank line.
- **t=16:** Snape silent ($posInCycle = 7$). Dumbledore's $posInCycle = (16 - 16) \bmod 8 = 0$, coincides with the start of state 0 $\rightarrow$ **Dumbledore**. CycleSeparator fires: blank line.
- **t=24:** Dumbledore fires again (same reasoning); blank line.
- After t=24: BOOM!

Additional Requirements

- Use packages; do not use the default package.
- Do not use `instanceof` or casting anywhere.
- State must be a Java **record**.
- `TickListener` must carry the `@FunctionalInterface` annotation.
- The `CycleSeparator` must be a **lambda** — no anonymous class, no separate class.

- Provide a UML class diagram (hw4.pdf).
- Write **JUnit** unit tests.

Bonus (+5 points each) - Maximal grade: 100

Unregisterable Listeners

Extend `Observable` with a third method:

API Specification

```
// Returns true if listener was found and removed; false otherwise.  
boolean unregister(TickListener listener);
```

Implement `unregister` in `PipeBomb`.

Then implement `FiniteEntity` — a `TickListener` that wraps another listener and an `Observable`. It counts firings; once the count reaches `maxFires`, it calls `observable.unregister(this)` and is never notified again.

Examples

```
// Ron fires at most 3 times, then silently disappears.  
TickListener ron = new StateMachineEntity("Ron", 1,  
    new State("Ron", 2), new State("Ron Weasly", 4));  
bomb.register(new FiniteEntity(ron, bomb, 3));
```

Demonstrate `FiniteEntity` in `Main` (or a separate `BonusMain`). Verify by inspection that the listener stops firing after the declared number of activations.

Conditional Entity

Implement `ConditionalEntity` — a `TickListener` that replaces the fixed `startTick` with a `Predicate<Integer>`. On each tick, if the predicate returns `true`, the entity prints its message.

API Specification

```
public class ConditionalEntity implements TickListener {  
    public ConditionalEntity(Predicate<Integer> predicate,  
                            String message)  
  
    @Override  
    public void onTick(int tick)  
}
```

Register at least **two** `ConditionalEntity` instances using lambda predicates. For example:

Examples

```
// Prints on every tick divisible by 7.  
bomb.register(new ConditionalEntity(  
    t -> t % 7 == 0, "Dobby is free!"));  
  
// Prints on ticks 13 and 42.  
bomb.register(new ConditionalEntity(  
    t -> t == 13 || t == 42, "Avada Kedavra!"));
```

Submission Instructions

Submit a single archive named:

studentID1_studentID2.zip

API Specification

```
studentID1_studentID2.zip  
hw4.pdf      -- UML class diagram  
hw4.jar      -- source code, tests, and compiled class files
```

Verify your submission with:

```
java -jar hw4.jar    (default countdown 160)  
java -jar hw4.jar 24 (must match the sample output above)
```

